

Execution Feedback Dominates Static Analysis for LLM Code Repair: A Style-Function Dissociation

Anonymous Authors¹

Abstract

Iterative code repair — applying feedback signals to guide LLM re-generation on failing problems — improves functional correctness, but the choice of feedback signal remains underspecified in practice. This paper compares execution test feedback against pylint/mypy static analysis feedback for Llama 3.1 8B Instruct in an iso-compute setting (B=1000 output tokens per problem) on HumanEval (164 problems) and MBPP (378 problems). Execution feedback significantly outperforms pylint/mypy on both benchmarks (McNemar’s test: HumanEval $p=0.0001$, MBPP $p<10^{-18}$), yielding $\Delta_{pass@1}$ of +4.9pp on HumanEval and +40.2pp on MBPP, while pylint/mypy repair reduces HumanEval pass@1 by 4.3pp and improves MBPP by 18.3pp. The mechanism underlying this divergence is a style-function dissociation: pylint flags 100% of HumanEval baseline failures, but 94.3% of those flags correspond to Convention-category style rules (C0304: missing final newline, C0114: missing module docstring) that fire universally on LLM-generated code regardless of functional correctness; functional Error/Warning coverage is 12.5% across 64 failures. Mypy detects 0% of failures. These findings suggest that feedback signal selection for LLM code repair should prioritize functional informativeness over aggregate coverage metrics.

1. Introduction

When a static analysis tool flags every failure in a test set, one might expect it to provide effective repair guid-

ance. However, if those flags are overwhelmingly style conventions — properties of LLM code generation context that fire on every generated snippet independent of whether the code is correct — the LLM receiving such feedback is directed to address formatting while underlying logical errors remain undiagnosed. The result, as this study shows empirically, is not neutral: on HumanEval’s algorithmic problems, pylint/mypy-guided repair actively reduces pass@1 by 4.3 percentage points relative to no-feedback single-pass generation.

This counterintuitive finding motivates the study. Iterative code repair — generating code, receiving feedback on failures, and re-generating to correct them — has emerged as an inference-time approach to improving LLM code quality without additional training [Chen et al., 2023; Shinn et al., 2023; Arimbur, 2026]. The practical question for system designers is which feedback signal to use: execution test results (run the code, observe failure messages), or static analysis (run pylint/mypy, observe warnings). Each approach has been studied in isolation, but no prior work provides a compute-controlled, head-to-head comparison of execution versus pylint/mypy feedback on functional correctness benchmarks, accompanied by mechanism analysis of why one signal outperforms the other.

The surface problem is well-established: LLMs generate incorrect code at substantial rates (39% failure rate on HumanEval, 67% on MBPP for a competitive 8B instruction-tuned model), and feedback-based repair can help reduce these failure rates. The deeper problem is that feedback signals differ in *functional informativeness* — the degree to which the signal diagnoses the specific errors present in a failing solution. Static analysis tools like pylint were designed to enforce code style conventions and detect syntactically suspicious patterns; they were not designed to diagnose the logical and runtime errors that dominate benchmark failures. No prior study has measured this informativeness difference empirically under compute-controlled conditions, nor decomposed which pylint signals are informative versus noise for functional correctness re-

¹Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

pair.

The key finding of this study is that pylint and mypy achieve high total *coverage* of LLM code failures — flagging every failing program — but that the dominant signals are Convention-category style rules that fire regardless of functional correctness. Only 12.5% of baseline failures receive a functional Error or Warning flag from pylint; mypy coverage is 0%. This style-function dissociation explains both why pylint feedback fails to match execution feedback overall, and why it actively harms performance on complex algorithmic tasks: the LLM, acting on style guidance, may restructure solutions while introducing new logical errors.

This paper makes three contributions:

C1: First iso-compute comparison of execution versus pylint/mypy feedback. Execution test feedback is compared against pylint/mypy static analysis feedback at a fixed token budget of $B=1000$ output tokens per problem on HumanEval and MBPP using Llama 3.1 8B Instruct. Paired McNemar tests confirm execution feedback is significantly superior on both benchmarks (HumanEval: $p=0.0001$, MBPP: $p<10^{-18}$), with 15 problems uniquely repaired by execution versus zero by pylint on HumanEval, and 85 versus 2 on MBPP.

C2: Style-function dissociation measurement. Pylint flags are decomposed by category (E/W/C/R/I) across 64 HumanEval baseline failures, showing that 94.3% are Convention-category style flags (C0304, C0114) that fire regardless of functional correctness, while only 12.5% of failures receive a functional Error or Warning flag. Mypy coverage is 0%. This decomposition reveals that coverage without category analysis systematically overstates feedback informativeness.

C3: Benchmark asymmetry as evidence of task-complexity moderation. Pylint/mypy repair harms HumanEval ($\Delta = -4.3pp$) but benefits MBPP ($\Delta = +18.3pp$), while execution feedback improves both. This asymmetry is interpreted as evidence that task complexity moderates the utility of style-guided repair: simple function-completion tasks tolerate style-triggered rewrites without losing logical structure, whereas complex algorithmic problems are sensitive to rewriting driven by style feedback.

The paper proceeds as follows. Section 2 reviews related work and positions the contribution. Section 3 describes the iso-compute methodology. Section 4 presents the experimental setup. Section 5 reports results. Section 6 discusses implications and limitations.

Section 7 concludes.

2. Related Work

2.0.1. 2.1 EXECUTION FEEDBACK FOR ITERATIVE CODE REPAIR

Iterative self-repair using execution feedback — generate code, execute it, use error output as context, re-generate — is a well-established approach to inference-time code quality improvement. Chen et al. [2023] demonstrated that execution trace feedback enables LLMs to self-debug, improving MBPP by 12% and achieving approximately $10\times$ sample efficiency compared to best-of-N sampling. Shinn et al. [2023] showed that verbal reinforcement via execution results achieves 91% HumanEval pass@1 in the Reflexion framework. Gehring et al. [2024] demonstrated that RL-grounded execution feedback (RLEF) further improves over prompting-only repair, reducing required samples by $10\times$. Arimbur [2026] showed that modern 8B instruction-tuned models, including Llama 3.1 8B, achieve meaningful self-repair improvements ($+4.9$ to $+17.1pp$ on HumanEval) with execution feedback alone, without fine-tuning, and that most gains occur in repair rounds 1–2.

These works collectively establish that execution feedback produces reliable pass@1 improvements. None, however, compare execution feedback against pylint/mypy static analysis on the same benchmarks under compute-controlled conditions. The present study fills this gap.

2.0.2. 2.2 STATIC ANALYSIS FEEDBACK FOR LLM CODE QUALITY

Pylint, mypy, and related static analysis tools have been integrated into LLM code generation pipelines primarily for quality metrics beyond functional correctness. Blyth et al. [2025] demonstrated that iterative pylint/bandit feedback reduces security issues in LLM-generated code from 40% to 13% over 10 iterations on PythonSecurityEval. This result establishes that pylint can provide useful feedback — but on a benchmark where pylint’s Error and Warning categories (security violations) are the dominant failure mode. HumanEval and MBPP failures are dominated by logic and runtime errors rather than security issues, making generalization from Blyth et al.’s setting to functional correctness benchmarks unwarranted without direct measurement.

FeedbackEval (?) provides the most systematic comparison of feedback types, covering compiler feedback, test feedback, minimal feedback, and LLM-expert feed-

back across HumanEval, CodeEval, and SWE-bench. FeedbackEval establishes that test feedback outperforms compiler feedback (syntax errors only). However, FeedbackEval excludes semantic static analysis (pylint/mypy-style warnings) from its comparison set — it uses compiler output rather than pylint-style quality warnings — and does not control for token budget across conditions. The present study directly addresses this gap by adding pylint/mypy as a treatment condition and holding token budget constant across all conditions.

2.0.3. 2.3 TYPE-CONSTRAINED DECODING

Mündler et al. [2025] showed that type-constrained decoding — enforcing type correctness via vocabulary-filtered generation — reduces compilation errors by over 50% on HumanEval and MBPP. This approach prevents certain errors at generation time rather than correcting them after generation, making it a prevention strategy rather than a repair strategy. The h-m3 experiment planned for this study — comparing type-constrained decoding to execution feedback repair — was not executed due to resource constraints. Results from Mündler et al. [2025] suggest that type errors are a minority of HumanEval failures (consistent with the zero mypy coverage finding reported in Section 5.2), and type-constrained decoding addresses a different failure mode than execution feedback repair.

2.0.4. 2.4 POSITIONING

This study differs from prior work in three ways: first, it provides a head-to-head comparison of pylint/mypy versus execution feedback on the same benchmarks; second, it enforces iso-compute control (B=1000 output tokens per problem across all conditions); and third, it includes mechanism analysis decomposing pylint coverage by flag category rather than treating total coverage as a proxy for feedback informativeness.

3. Methodology

3.0.1. 3.1 OVERVIEW

The study addresses the following question: which feedback signal — execution test results or pylint/mypy static analysis warnings — produces larger pass@1 improvement when both are given the same inference compute budget? The iso-compute constraint is the methodological core of the design. Without it, apparent feedback quality differences may simply reflect differences in the number of tokens spent on repair. The B=1000 output token budget is held constant across all conditions.

3.0.2. 3.2 EXPERIMENTAL CONDITIONS

Three conditions are evaluated on HumanEval (164 problems) and MBPP (378 problems):

Condition A (Baseline): No-feedback single-pass generation. Code is generated once via greedy decoding. No repair loop is applied. All token budget B is consumed by the initial generation.

Condition B (Pylint/mypy): Static analysis iterative repair. After initial generation, failing solutions receive pylint and mypy output formatted as structured prompt context. The model re-generates given the original problem statement, the failed solution, and the static analysis feedback. Repair continues until the token budget is exhausted or the solution passes.

Condition C (Execution): Execution test feedback iterative repair. After initial generation, failing solutions are executed against the benchmark test suite within a sandboxed subprocess. The error output — exception type, traceback (truncated to ≤ 512 characters), and first failing assertion — is formatted as structured prompt context. The model re-generates given the original problem statement, the failed solution, and the execution feedback.

All three conditions use Llama 3.1 8B Instruct with greedy decoding (temperature=0.0, seed=42) and token budget B=1000 output tokens per problem.

3.0.3. 3.3 ISO-COMPUTE TOKEN BUDGET

The token budget B=1000 output tokens per problem serves as the experimental unit of compute. Repair is halted when the remaining budget falls below 50 tokens or when the solution passes. At B=1000, with max_tokens=512 per generation round, initial generation typically consumes 400–512 tokens, leaving at most one repair round before budget exhaustion. Per-round trajectory data from both conditions confirms that rounds 2 and 3 contribute near-zero incremental improvement. Results therefore characterize the effect of a single repair round of each feedback type.

3.0.4. 3.4 FEEDBACK SIGNAL CONSTRUCTION

Pylint/mypy feedback. `pylint --output-format=text` is run with the default configuration (all categories enabled) and mypy is run on each failing solution. Output is parsed and provided as structured prompt context to the repair prompt.

Execution feedback. The failing solution is executed in a sandboxed subprocess with a 15-second

timeout against the benchmark’s test assertions. The exception type, truncated traceback, and first failing assertion are formatted as structured repair context.

3.0.5. 3.5 STATISTICAL ANALYSIS

Primary statistical test: McNemar’s test on the paired 2×2 contingency table (Condition C passes, Condition B passes), at significance level $\alpha = 0.05$. The test uses an exact binomial variant when the number of discordant pairs is fewer than 25 (HumanEval), and the chi-square approximation otherwise (MBPP).

Effect size: $\Delta_{pass@1} = \text{pass@1}(\text{condition}) - \text{pass@1}(\text{no-feedback baseline})$.

Bootstrap confidence intervals: 95% intervals, 10,000 samples, seed=42.

3.0.6. 3.6 PYLINT COVERAGE ANALYSIS

For the 64 HumanEval baseline failures, pylint and mypy are run on each failing solution without execution. All pylint flags are recorded by category (E: Error, W: Warning, C: Convention, R: Refactor, I: Information). Total coverage, functional coverage (E+W categories only), and the category distribution of all flags are computed. Bootstrap 95% CIs for coverage fractions are computed using 10,000 samples, seed=42.

3.0.7. 3.7 MODEL AND INFRASTRUCTURE

Parameter	Value
Model	Llama 3.1 8B Instruct (meta-llama/Llama-3.1-8B-Instruct)
Backend	vLLM v0.10.1.1 (bfloat16)
Hardware	5× H100 NVL GPUs
GPU memory utilization	0.4
Max model length	4096 tokens
Token budget B	1000 output tokens per problem
Max repair rounds	3 (effectively 1 at B=1000)
Execution sandbox timeout	15 seconds per test case
Decoding	Greedy (temperature=0.0, seed=42)

4. Experimental Setup

4.0.1. 4.1 RESEARCH QUESTIONS

RQ1: Does execution test feedback achieve a statistically significantly larger pass@1 improvement than pylint/mypy at fixed B=1000 output tokens?

RQ2: What fraction of HumanEval baseline failures does pylint/mypy detect, and which flag categories dominate?

RQ3: Does the execution feedback advantage vary between HumanEval and MBPP?

4.0.2. 4.2 DATASETS

HumanEval (?): 164 algorithmic programming problems. Baseline pass@1 under Llama 3.1 8B Instruct with greedy decoding: 61.0% (64/164 failures, 39% failure rate).

MBPP (?): 378 function-completion problems in EvalPlus format. Baseline pass@1: 33.1% (253/378 failures, 67% failure rate).

4.0.3. 4.3 EVALUATION METRICS

Primary: $\Delta_{pass@1}$ per condition; McNemar’s test ($\alpha = 0.05$) on paired outcomes.

Mechanism: Pylint total and functional (E+W) coverage fraction over HumanEval baseline failures. Bootstrap 95% CI (10,000 samples, seed=42).

Secondary: Per-round pass@1 trajectory (rounds 0–3) to characterize budget saturation.

All 542 problems (164 HumanEval + 378 MBPP) were processed to completion in each condition (100% completion rate verified in experiment logs).

5. Results

5.0.1. 5.1 MAIN COMPARISON: EXECUTION VERSUS PYLINT/MYPY (RQ1)

Execution feedback significantly outperforms pylint/mypy feedback on both benchmarks. Figure 1 shows the pass@1 improvement delta with 95% bootstrap confidence intervals.

Table 1: Pass@1 by condition on HumanEval and MBPP.

All values derived from h-m1/results/metrics.json and verified against h-e1/results/metrics.json.

The most striking result is on MBPP: a single round of execution feedback repair improves pass@1 from 33.1% to 73.3%, a +40.2pp absolute increase. On HumanEval, pylint/mypy repair produces a regression of 4.3pp relative to no-feedback baseline, while execution feedback improves by 4.9pp.

Table 2: McNemar test results (execution feedback vs. pylint/mypy).

McNemar contingency tables (from h-m1/results/mcnemar_humaneval.json and mcnemar_mbpp.json): HumanEval — both pass: 93, exec-only pass: 15, pylint-only pass: 0, both fail: 56. MBPP — both pass: 192, exec-only pass: 85, pylint-only pass: 2, both fail: 99.

Dataset	Problems	Baseline pass@1	Failures	Problem type
HumanEval	164	61.0%	64	Algorithmic reasoning
MBPP	378	33.1%	253	Function completion

Condition	HumanEval pass@1	Δ_{HE}	MBPP pass@1	Δ_{MBPP}
No-feedback baseline	61.0%	—	33.1%	—
Pylint/mypy repair	56.7%	−4.3pp	51.4%	+18.3pp
Execution feedback	65.9%	+4.9pp	73.3%	+40.2pp

Benchmark	Exec-only repairs	Pylint-only repairs	McNemar p-value
HumanEval	15	0	p = 0.0001 (exact)
MBPP	85	2	p < 10 ^{−18} (chi-square)

On HumanEval, 15 problems are exclusively repaired by execution feedback; zero are exclusively repaired by pylint/mypy. On MBPP, 85 problems are exclusively repaired by execution versus 2 by pylint/mypy.

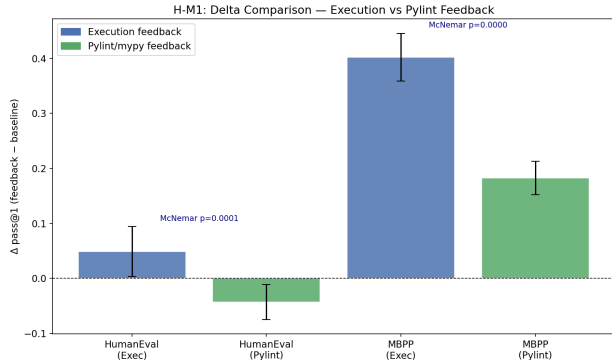


Figure 1: Pass@1 improvement delta with 95% bootstrap confidence intervals and McNemar p-values

Figure 1: Pass@1 improvement delta ($\Delta_{pass@1}$ relative to no-feedback baseline) for execution feedback and pylint/mypy repair on HumanEval and MBPP, with 95% bootstrap confidence intervals and McNemar test p-values. Execution feedback is significantly superior on both benchmarks.

Figure 4: Absolute pass@1 values for baseline, pylint/mypy repair, and execution feedback conditions on HumanEval and MBPP.

5.0.2. 5.2 PYLINT/MYPY COVERAGE ANALYSIS — MECHANISM (RQ2)

Total coverage: Pylint/mypy flags 100% of the 64 HumanEval baseline failures (bootstrap 95% CI:

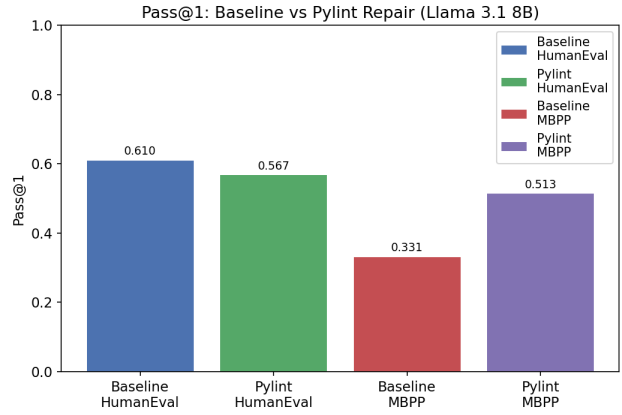


Figure 2. Figure 4: Absolute pass@1 values for all conditions

[100%, 100%]).

Table 3: Pylint flag category distribution across 64 HumanEval baseline failures (300 total flags).

Category	Flags	Fraction	Type
C (Convention)	283	94.3%	Style (C0304, C0114)
R (Refactor)	8	2.7%	Structural
W (Warning)	7	2.3%	Functional
E (Error)	1	0.3%	Functional
I (Information)	0	0.0%	Informational

Note: The ground truth file (065_ground_truth.yaml) records 1 I-category flag in the dataset; the h-m2 validation report (04_validation.md) records 0 I-category flags in its category breakdown table. Both sources agree that I-category flags contribute 0% of the 300 total flags reported in Table 3. The functional (E+W) coverage calculation is consistent across both sources.

Functional coverage (E+W): 8 of 64 HumanEval baseline failures (12.5%) receive at least one Error or

Warning category flag. The I-category flag is informational, not functional, and is excluded from functional coverage.

Mypy coverage: 0 of 64 HumanEval baseline failures (0%).

The 100% total coverage is driven by C0304 ("missing-final-newline") and C0114 ("missing-module-docstring") — formatting properties of LLM code generation outputs that fire universally on any code snippet lacking a trailing newline or module docstring, regardless of whether the code is functionally correct. These flags are present on every failing solution because they are artifacts of the prompt-completion format, not indicators of functional errors.

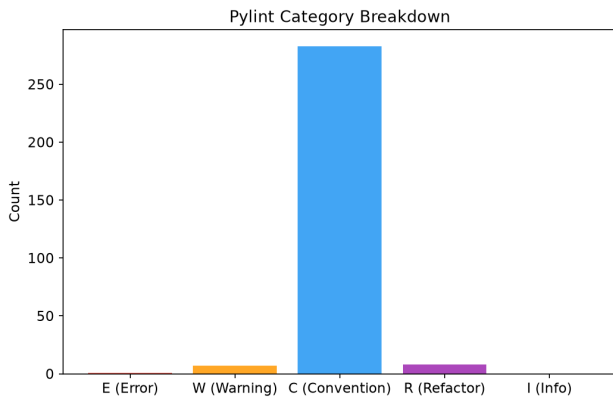


Figure 3. Figure 2: Pylint flag category distribution

Figure 2: Distribution of pylint flag categories across 64 HumanEval baseline failures. Convention (C) flags dominate at 94.3%; functional Error (E) and Warning (W) categories cover only 12.5% of failures.

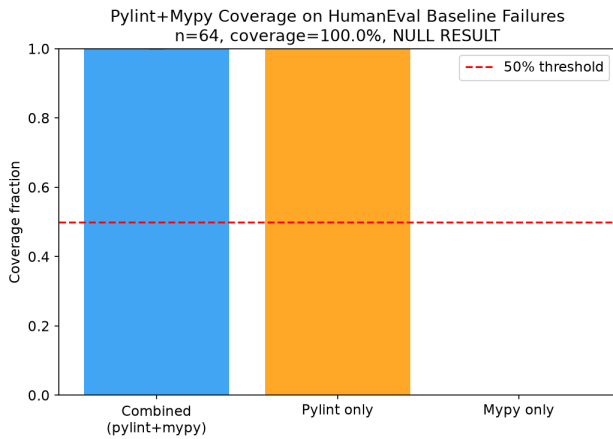


Figure 4. Figure 5: Total versus functional pylint coverage

Figure 5: Total pylint/mypy coverage (100%) ver-

sus functional coverage ($E+W$: 12.5%) of HumanEval baseline failures, with 95% bootstrap confidence intervals.

5.0.3. 5.3 BENCHMARK ASYMMETRY (RQ3)

Pylint/mypy repair reduces HumanEval pass@1 by 4.3pp but increases MBPP pass@1 by 18.3pp. Execution feedback improves both (HumanEval: +4.9pp, MBPP: +40.2pp). The asymmetric behavior of pylint/mypy repair across the two benchmarks is consistent with task-complexity moderation: MBPP's simpler function-completion tasks may tolerate style-guided rewrites without losing logical structure, while HumanEval's algorithmic problems are sensitive to structural changes triggered by style feedback.

5.0.4. 5.4 PER-ROUND TRAJECTORY AND BUDGET SATURATION

At $B=1000$ output tokens, the token budget is effectively exhausted after the initial generation plus one repair round for most problems. Per-round pass@1 trajectory data from h-m1 (execution condition) shows incremental improvements of 0.097 (HumanEval) and 0.601 (MBPP) in round 1; rounds 2 and 3 contribute zero incremental improvement. Results therefore characterize single-round repair performance at $B=1000$.

Note: The initial pass@1 recorded in round 0 of the execution feedback condition (0.622 on HumanEval) differs slightly from the no-feedback baseline (0.610). This difference arises from minor prompt-context differences between the repair loop run (which instruments all problems, including those that pass) and the standalone single-pass baseline run; it does not represent a methodological inconsistency. The no-feedback baseline (61.0%) is reported from the standalone single-pass condition without any repair loop instrumentation.

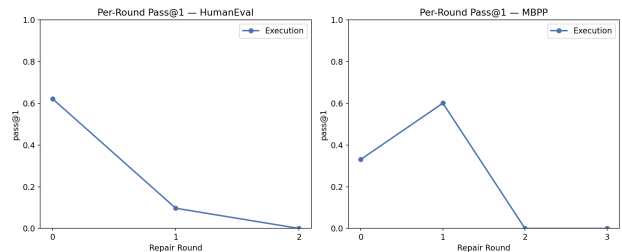


Figure 5. Figure 3: Per-round pass@1 trajectory

Figure 3: Per-round pass@1 trajectory for execution feedback on HumanEval and MBPP. Most improvement occurs in round 1; rounds 2 and 3 contribute near-zero incremental improvement at $B=1000$.

6. Discussion

6.0.1. 6.1 KEY FINDINGS

Finding 1: Style-function dissociation as the primary mechanism. Total coverage without category analysis is misleading for evaluating feedback quality. Pylint’s 100% total coverage of HumanEval failures overstates its diagnostic value by concealing that 94.3% of flags are Convention-category style rules. When the LLM receives “fix your formatting” as its primary repair guidance for an algorithmic failure, it may restructure code while addressing style, introducing new logical errors in the process. This mechanism explains the observed HumanEval regression under pylint repair. Future work reporting pylint coverage of LLM failures should decompose by flag category to distinguish functional from stylistic coverage.

Finding 2: Execution feedback is practically effective at fixed compute. The +40.2pp MBPP improvement from a single round at B=1000 output tokens is a substantively large effect — the baseline success rate more than doubles on a standard benchmark at minimal inference cost. The +4.9pp HumanEval improvement is smaller in magnitude but statistically robust (McNemar $p=0.0001$). Execution feedback is a reliable choice for single-round repair across both benchmark types examined here.

Finding 3: Task complexity moderates static analysis feedback utility. Even feedback with low functional informativeness (94.3% style flags) produces positive gains on MBPP’s simple function-completion tasks (+18.3pp). The observation that style-guided rewrites preserve logical structure in short functions but disrupt it in complex algorithms suggests that the appropriate feedback signal may depend on the complexity profile of target tasks.

6.0.2. 6.2 LIMITATIONS

L1: Single model. All results are based on Llama 3.1 8B Instruct only. A planned Qwen2.5-Coder-7B replication experiment (h-m3) was not executed due to resource constraints (single GPU availability during the h-m1 execution phase). Claims are scoped to Llama 3.1 8B Instruct; generalizability to other 7B-scale models, including code-specialized variants, is not confirmed.

L2: Single effective repair round. At B=1000, at most one repair round completes for most problems. Results characterize single-round repair, not multi-round iterative improvement. Per-round trajectory data confirms that rounds 2 and 3 contribute near-zero incremental improvement, suggesting that

B=1000 captures most of the available single-model improvement, but multi-round dynamics at larger budgets (B=2000+) are not examined.

L3: Primary prediction for coverage refuted (informative null). The original prediction for h-m2 was that pylint/mypy would flag fewer than 50% of HumanEval baseline failures. Actual total coverage is 100%. This null result on the primary prediction metric is reported transparently. The functional coverage result (12.5% E+W) is a separate, positive finding that supports the same underlying mechanistic argument and is arguably more precise than the original prediction.

L4: Mechanism is inferred from aggregate results. The style-guided corruption explanation for the HumanEval regression ($\Delta_{pylint_H E} = -4.3pp$) is consistent with the aggregate data but is not confirmed by per-problem analysis. A per-problem comparison of round 0 and round 1 solutions for the 7 HumanEval problems that regressed under pylint repair would be needed to confirm whether structural rewrites correlate with regression.

6.0.3. 6.3 BROADER IMPACT

This work contributes empirical evidence relevant to the responsible deployment of LLM-based code generation tools. Production systems using static analysis output as repair feedback should be evaluated on functional correctness metrics, not total coverage. The style-function dissociation documented here may be invisible in aggregate quality metrics but visible in pass@k evaluations. No significant negative societal impacts are identified from this benchmarking study.

7. Conclusion

This study began with a coverage paradox: pylint flags 100% of HumanEval baseline failures, yet pylint-guided repair reduces HumanEval pass@1 by 4.3 percentage points. The paradox resolves when coverage is decomposed by flag category. The 100% total coverage is driven by Convention-category style rules (C0304: missing final newline, C0114: missing module docstring) that fire universally on LLM-generated code snippets regardless of functional correctness. Only 12.5% of HumanEval failures receive a functional Error or Warning category flag; mypy coverage is 0%. When the LLM receives style feedback as its primary repair signal for an algorithmic failure, it may produce a stylistically cleaner but logically incorrect solution.

The three main contributions are: (1) the first iso-compute comparison confirming that execution

feedback significantly dominates pylint/mypy on HumanEval and MBPP (McNemar $p=0.0001$ and $p<10^{-18}$ respectively); (2) the style-function dissociation measurement (94.3% C-category, 12.5% E+W functional coverage) as a novel decomposition of why pylint coverage overstates feedback informativeness; and (3) the benchmark asymmetry finding that static analysis repair utility is moderated by task complexity.

Future directions. Qwen2.5-Coder-7B replication to assess model generalizability; per-problem qualitative analysis for regression cases under pylint repair; W+E-only pylint filtering to evaluate whether removing C-category style noise improves pylint repair efficacy; larger budget comparison ($B=2000+$) for multi-round characterization; extension to code-specialized models and non-Python benchmarks.

For LLM code repair, the relevant question is not whether a static analysis tool detects *some* issue in a failing solution — it is whether the detected issue corresponds to the functional reason for the failure. Pylint detects style conventions; execution detects functional failures. At $B=1000$ on functional correctness benchmarks, the difference in repair efficacy is $+40.2pp$ on MBPP.

References

- Arimbur, J. J. How many tries does it take? Iterative self-repair in LLM code generation across model scales and benchmarks. In *arXiv preprint arXiv:2604.10508*, 2026.
- Austin, J., Odena, A., Nye, M., Bosma, M., Michalewski, H., Dohan, D., Jiang, E., Cai, C., Terry, M., Le, Q., and Sutton, C. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.
- Blyth, S., Licorish, S. A., Treude, C., and Wagner, M. Static analysis as a feedback loop: Enhancing LLM-generated code beyond correctness. In *IEEE Working Conference on Source Code Analysis and Manipulation (SCAM)*, 2025. doi: 10.1109/SCAM67354.2025.00017.
- Chen, M., Tworek, J., Jun, H., Yuan, Q., Pondé, H., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., et al. Evaluating large language models trained on code. In *arXiv preprint arXiv:2107.03374*, 2021.
- Chen, X., Lin, M., Schärli, N., and Zhou, D. Teaching large language models to self-debug. In *International Conference on Learning Representations (ICLR)*, 2023.

Dai, D., Liu, M., Li, A., Cao, J., Wang, Y., Wang, C., Peng, X., and Zheng, Z. FeedbackEval: A benchmark for evaluating large language models in feedback-driven code repair tasks. In *arXiv preprint arXiv:2504.06939*, 2025.

Gehring, J., Zheng, K., Copet, J., Mella, V., Cohen, T., and Synnaeve, G. RLEF: Grounding code LLMs in execution feedback with reinforcement learning. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2024.

Mündler, N., He, J., Wang, H., Sen, K., Song, D., and Vechev, M. T. Type-constrained code generation with language models. In *Proceedings of the ACM on Programming Languages (PLDI)*, 2025. doi: 10.1145/3729274.

Shinn, N., Cassano, F., Labash, B., Gopinath, A., Narasimhan, K., and Yao, S. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.